Machine Learning for Network Intrusion Detection with NSL-KDD

Northeastern University
Course: AAI 6600 — Applied Artificial Intelligence
Professor: Dr. Seshadri
Students: Will Daly, Wei Dong, Zahra Joulaei
Date: April 7, 2026

Real-World Problem

Enterprise networks generate far more traffic than a human analyst can review manually. Security teams need a way to identify suspicious connections quickly enough to stop denial-of-service attacks, probes, and unauthorized access before the damage spreads. This is a strong machine-learning problem because the traffic is high-volume, the feature space mixes categorical and numeric signals, and the relationships between variables are not always obvious from simple rules alone.

This project studies whether machine-learning models can distinguish normal network traffic from malicious traffic in a way that is useful for a security operations team. The practical business question is straightforward: can the organization catch more attacks while keeping false alarms low enough that analysts can still trust and use the system?

Dataset Selection And Rationale

The group selected the **NSL-KDD** benchmark dataset, using the local submission copy in `data/nsl-kdd/` and the Kaggle mirror maintained at Kaggle. NSL-KDD is a cleaned and improved version of the original KDD Cup 1999 intrusion-detection dataset and remains a common baseline for network intrusion research (Tavallaee et al., 2009).

We chose this dataset for four reasons:

1. It directly matches the course theme of malware and network intrusion detection.
2. It provides labeled train and test files, which supports an honest held-out evaluation.
3. It contains 41 traffic features plus labels, giving enough signal for both EDA and supervised learning.
4. It is widely cited, which makes it a credible benchmark for a classroom project.

The combined dataset contains **148,517** rows: **125,973** in `KDDTrain+` and **22,544** in `KDDTest+`. Each record represents a network connection and includes protocol, service, TCP flag, byte counts, login indicators, error rates, and host-level aggregate statistics.

EDA Workflow And Findings

What We Did In Exploration

The project loads the training and test files with explicit NSL-KDD column names, combines them for summary statistics, and then explores label frequencies, service distribution, connection flags, and representative numeric features. Attack labels are mapped to the five standard NSL-KDD categories — Normal, DoS, Probe, R2L, and U2R — using a comprehensive lookup table. The notebooks `EDA.ipynb` and `01_eda_nsl_kdd.ipynb` handle the visual exploration — `EDA.ipynb` provides the primary styled analysis used for the class presentation, while `01_eda_nsl_kdd.ipynb`

documents a full methodical walkthrough of the complete training set including fine-grained attack-type breakdowns. `main.py` reproduces the key EDA artifacts into `outputs/` for submission.

Data Volume And Structure

- Rows: **148,517**
- Predictive features: **41**
- Additional fields: `label` and `difficulty_level`
- Splits used: the dataset's original train/test files

Missing Data, Duplicates, And Cleanliness

The combined summary shows **0 missing values** and **0 duplicate rows**. That allowed the project to focus more on feature encoding and class balance than on record repair.

Outliers And Suspicious Values

Several numeric variables, especially `duration` and `src_bytes`, are strongly right-skewed. These values were not automatically removed as outliers because in intrusion detection they can be meaningful indicators of attack behavior rather than bad data. The boxplot visualizations below show distributions across all five attack categories, making the per-category skew patterns visible.

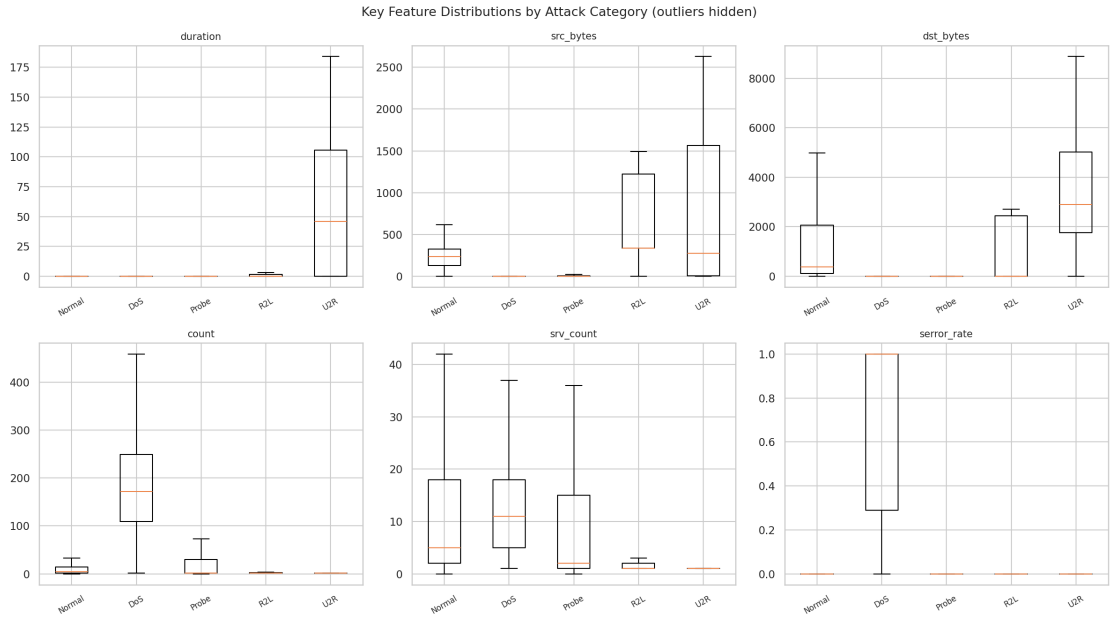

Figure 1: Key numeric feature distributions by attack category

Cleaning And Preparation Decisions

- Assigned explicit schema names to the raw text files
- Excluded `difficulty_level` from modeling because it is benchmark metadata rather than a real traffic feature
- Mapped 23 fine-grained attack labels to 5 standard categories (Normal, DoS, Probe, R2L, U2R) for multiclass analysis

- Converted the multiclass attack labels into a binary target for the primary classification pipeline: `normal = 0`, `attack = 1`
- One-hot encoded the categorical fields `protocol_type`, `service`, and `flag`
- Median-imputed and standardized the numeric columns

What We Found

EDA surfaced a clear class-imbalance pattern: normal traffic and a few high-volume DoS attacks dominate the dataset, while R2L and U2R attack types are extremely rare. The attack category distribution chart below shows Normal and DoS together make up over 85% of training data, with U2R representing less than 0.1%.

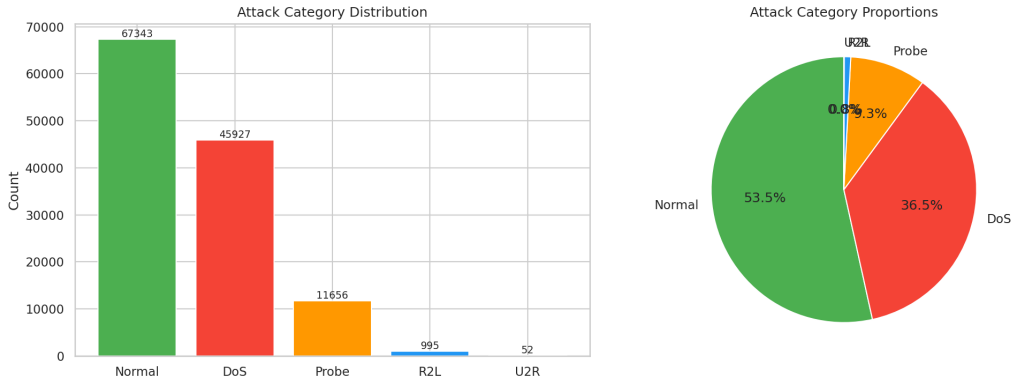

Figure 2: Attack category distribution — bar and donut charts of the five NSL-KDD categories

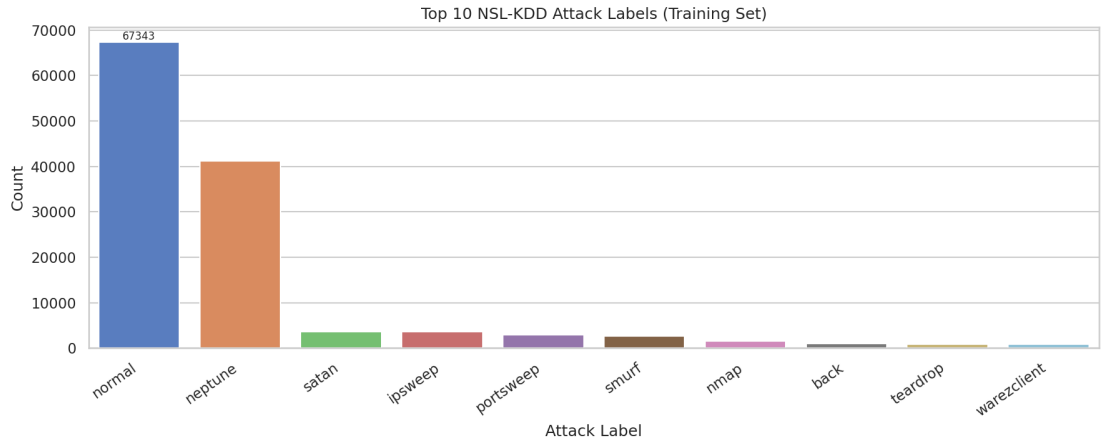

Figure 3: Fine-grained label distribution — top 10 individual attack types by record count

The correlation heatmap below revealed several strongly correlated feature pairs — such as `serror_rate` and `srv_serror_rate`, and `dst_host_serror_rate` and `dst_host_srv_serror_rate` — suggesting redundancy that could be addressed with feature selection in future iterations. The categorical feature analysis showed that protocol type, service, and TCP flag each carry useful separation signal between attack categories.

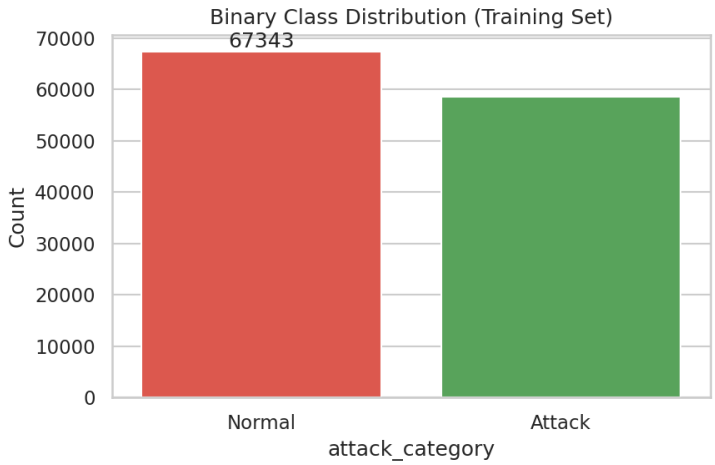

Figure 4: Binary class distribution — normal vs. attack

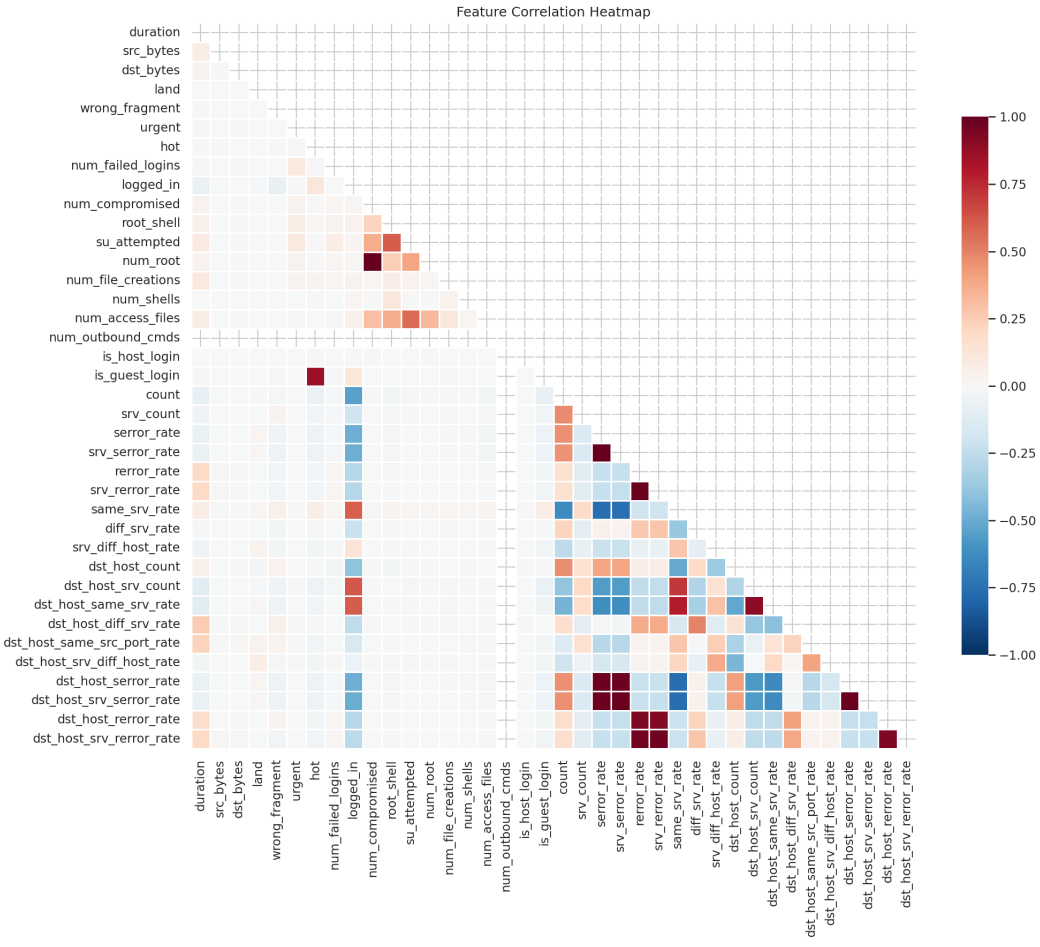

Figure 5: Feature correlation heatmap

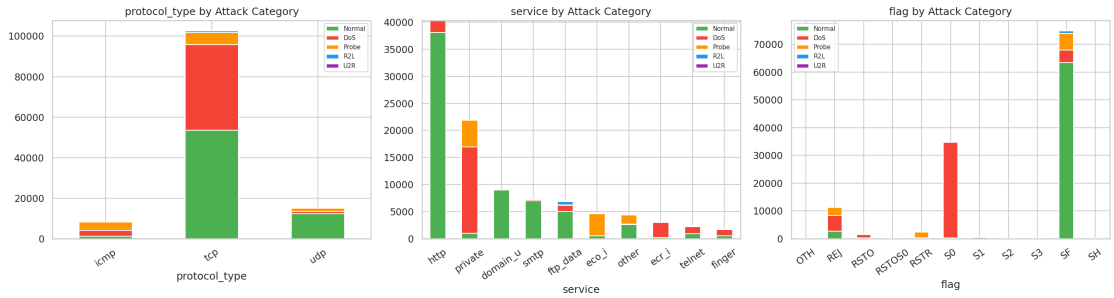

Figure 6: Categorical features (protocol type, service, flag) broken down by attack category

Proposed Next Steps From EDA

The exploration suggested four modeling priorities:

1. Start with a binary classifier before attempting fine-grained attack-type prediction.
2. Compare an interpretable linear model, a tree-based ensemble, and a gradient boosting model.
3. Use class-weighted models to address the imbalance problem directly.
4. Report precision, recall, F1, AUC, and confusion matrices so the class imbalance does not hide weak attack detection.

Machine Learning Methodology

Algorithms Used And Why

The final pipeline compares three supervised models with class-imbalance handling:

1. **Logistic Regression** (`class_weight='balanced'`): a strong interpretable baseline that is fast to train and easy to explain. Class weighting ensures the model does not ignore minority attack classes.
2. **Random Forest** (`class_weight='balanced'`, 300 estimators): an ensemble model that captures nonlinear interactions across the mixed feature space. The balanced class weights upweight rare attack samples during tree construction.
3. **Gradient Boosting** (100 estimators, subsample=0.8): a sequential boosting approach that builds each tree to correct the errors of the previous one, often achieving the strongest overall performance on structured datasets.

These models were selected because together they span the tradeoff between interpretability, robustness, and predictive power. Adding class weighting directly addresses the imbalance identified in EDA.

Metrics Used

The project evaluates each model on the held-out `KDDTest+` file using:

- Accuracy
- Precision
- Recall
- F1 score
- AUC (area under the ROC curve)

- Confusion matrix

These metrics align with the business problem. Security teams care about **recall** because missed attacks are costly, but they also care about **precision** because too many false positives create alert fatigue. AUC provides a threshold-independent view of overall model discrimination.

Code Walk-Through

The code is organized so each stage of the workflow is easy to explain in class:

- `src/data_loader.py` resolves the dataset path, supports offline local data, and loads `KDDTrain+` and `KDDTest+`
- `src/eda.py` computes dataset-level summary statistics, maps attack categories, and exports 6 sets of chart artifacts including correlation heatmaps and per-category feature distributions; supplementary full-dataset exploration is documented in `01_eda_nsl_kdd.ipynb`
- `src/train_models.py` builds the preprocessing pipeline with `ColumnTransformer`, fits three binary classifiers with class weighting, generates ROC curves, confusion matrix heatmaps, feature importance plots, and a model comparison chart. It also trains a separate multiclass Random Forest for 5-class attack-category prediction.
- `main.py` ties the full workflow together so a single command reproduces all project outputs

Results And Analysis

Binary Classification

The three models were evaluated on **22,544** held-out network connections.

Model	Accuracy	Precision	Recall	F1	AUC
Logistic Regression	0.7543	0.9169	0.6250	0.7433	0.7915
Random Forest	0.7663	0.9677	0.6098	0.7481	0.9623
Gradient Boosting	**0.8204**	**0.9704**	**0.7059**	**0.8173**	**0.9570**

Binary Confusion Matrices

ROC Curves

Random Forest and Gradient Boosting both achieve excellent AUC scores above 0.95, while Logistic Regression lags at 0.79 — confirming that the nonlinear relationships in the data reward more flexible models.

Feature Importance

The top 20 predictive features from Random Forest are shown below. Service-related one-hot features, error rates (`serror_rate`, `srv_serror_rate`), and connection-level statistics (`src_bytes`, `dst_host_srv_count`) rank highest, confirming the patterns observed during EDA.

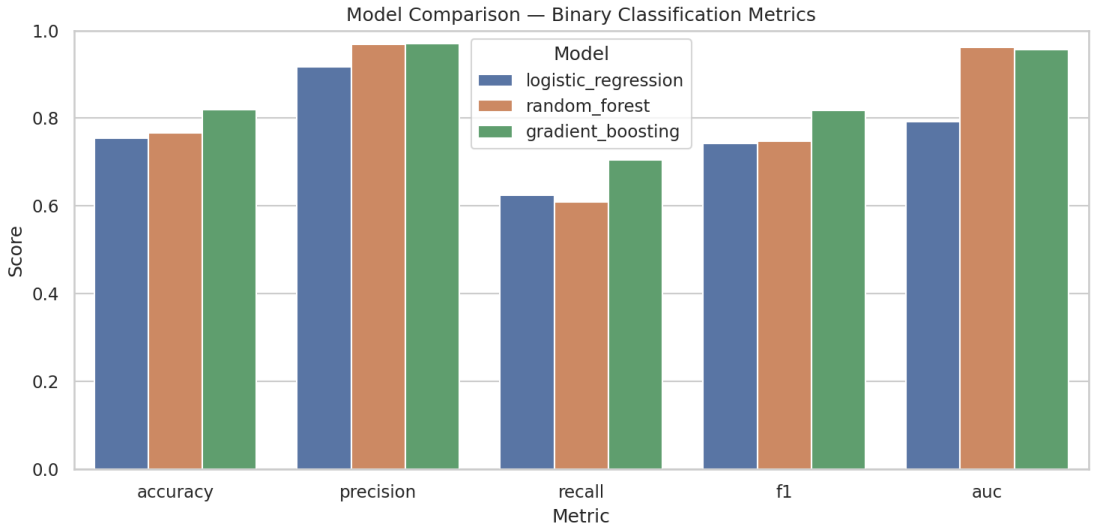

Figure 7: Model comparison — accuracy, precision, recall, F1, and AUC across all three classifiers

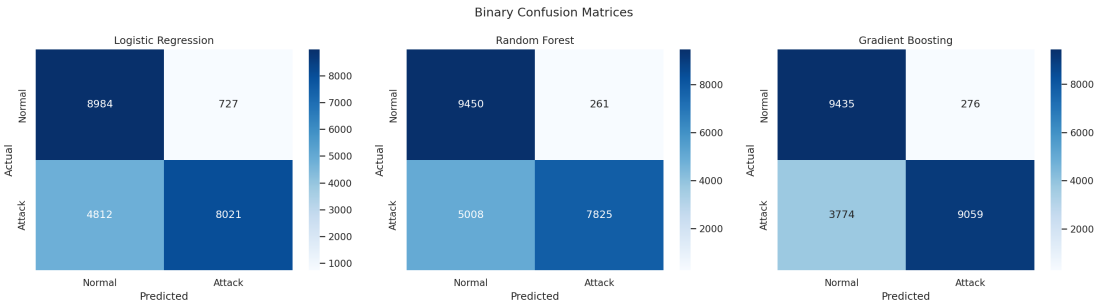

Figure 8: Side-by-side confusion matrices for all three binary classifiers

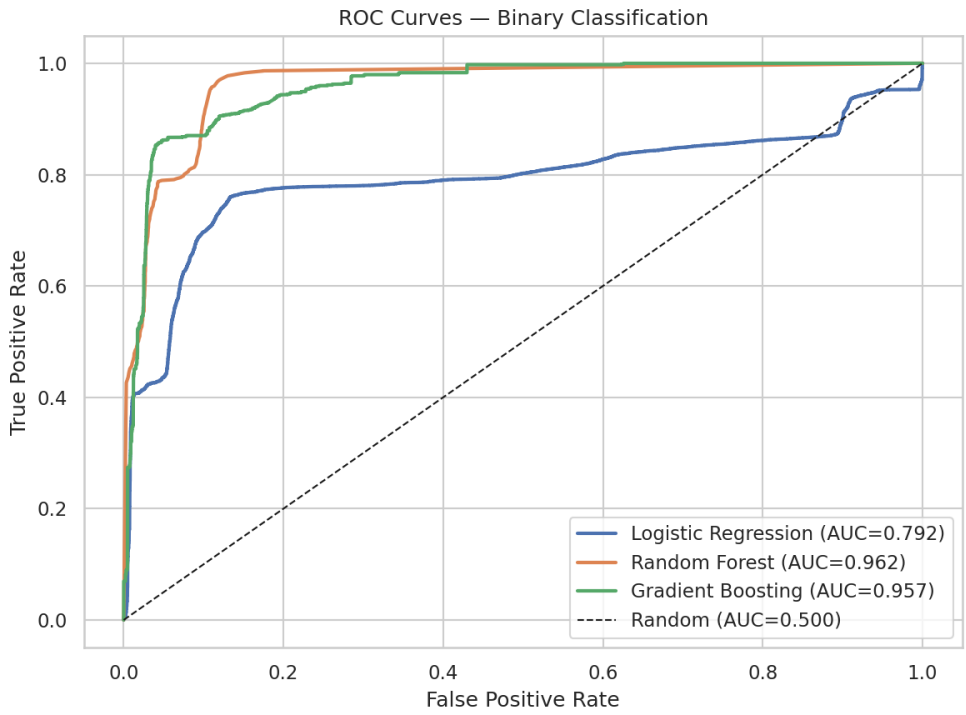

Figure 9: ROC curves with AUC scores for all three models

Multiclass Attack-Category Classification

A separate balanced Random Forest was trained on the five standard NSL-KDD categories:

Category	Precision	Recall	F1	Support
Normal	0.63	0.97	0.77	9,711
DoS	0.96	0.76	0.85	7,458
Probe	0.85	0.59	0.70	2,421
R2L	0.81	0.00	0.01	2,887
U2R	0.50	0.03	0.06	67

Overall multiclass accuracy: **0.7363**

The multiclass results expose a critical insight: while the model handles Normal and DoS traffic well, **R2L and U2R attacks are nearly undetectable** with standard classification approaches. A breakdown of the training data explains why — the combined R2L and U2R categories contain only **1,047 records out of 125,973 (0.83%)**, with U2R accounting for just 52 examples across four attack types (`buffer_overflow`: 30, `rootkit`: 10, `loadmodule`: 9, `perl`: 3) and R2L totaling 995 examples dominated by `warezclient` (890) with the remaining seven types having fewer than 55 records each. With so few examples, the model cannot learn reliable decision boundaries for these classes. This finding directly informs the prescriptive recommendations below.

Interpretation

Gradient Boosting performed best overall on binary classification, achieving the highest accuracy (0.82), recall (0.71), and F1 (0.82). Its key advantage is substantially better recall than Logistic Regression or Random Forest, meaning it catches more actual attacks while maintaining near-perfect precision. The ROC curves confirm both tree-based models achieve strong discrimination (AUC > 0.95).

The multiclass analysis reveals that binary detection is the more practical deployment approach;

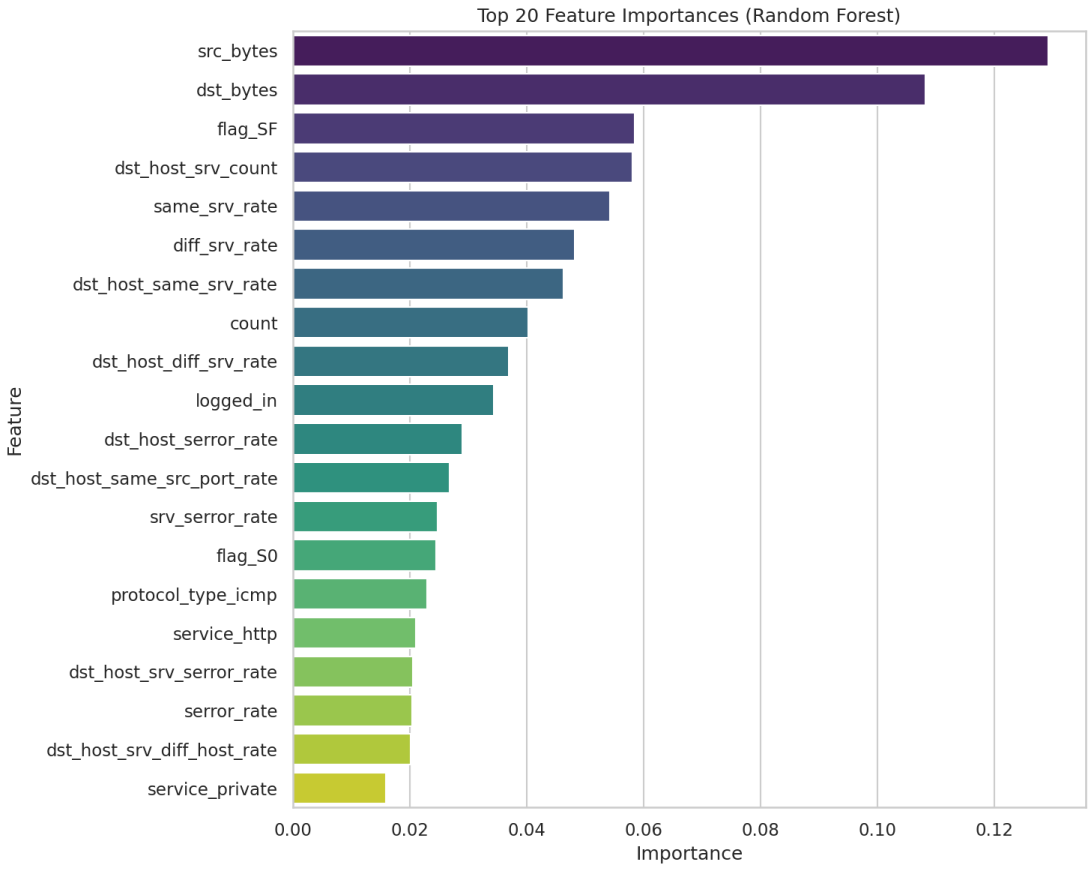

Figure 10: Top 20 Random Forest feature importances

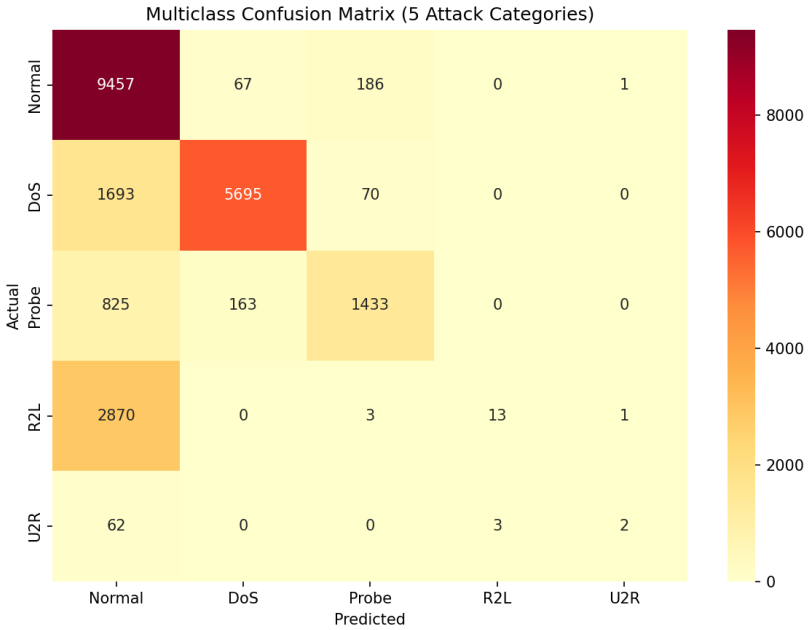

Figure 11: Multiclass confusion matrix — 5-category attack classification

1. **Deploy Gradient Boosting as the initial detection model** for analyst review queues, given its superior recall and F1.
2. **Tune the decision threshold** to target higher operational recall when attack coverage matters more than alert volume. The ROC curves show both tree-based models maintain strong precision across a wide range of thresholds.
3. **Prioritize monitoring of service, error-rate, and connection-count features** because they carry the strongest predictive signal per the feature importance analysis.
4. **Invest in specialized detection for R2L and U2R attacks**, which the multiclass analysis showed are nearly invisible to standard classifiers. Techniques like anomaly detection, SMOTE oversampling, or cost-sensitive learning should be evaluated.
5. **Establish feedback loops** between model predictions and analyst outcomes to continuously refine the system — a key step in the MLOps Maintain and Monitor phases.
6. **Extend to multiclass prediction** after improving rare-attack detection, so the system can route different attack types to different response workflows.

APLC Framework Alignment

This project follows the four quadrants of Dr. Seshadri's Analytics Product Life Cycle (APLC):

- **Business Quadrant**: We framed the business problem (network intrusion detection), right-sized the project scope to the NSL-KDD benchmark, and organized the team with clear roles and weekly milestones.
- **Data Engineering Quadrant**: We conducted comprehensive EDA including feature correlation analysis, attack-category mapping, class-imbalance assessment, and categorical feature analysis across 6 visualization sets.
- **Modeling Quadrant**: We developed and compared three ML algorithms with class weighting, evaluated using multiple metrics, and extended to multiclass prediction. We iterated based on initial results to add Gradient Boosting after finding the first two models had limited recall.
- **Software Engineering Quadrant**: We packaged the project as a reproducible Python pipeline with modular source code, automated outputs, and a single-command entry point.

Conclusion

This project shows that machine learning can support intrusion detection in a practical and defensible way. On the NSL-KDD benchmark, Gradient Boosting offered the best operating tradeoff for a real-world security team, achieving 0.82 accuracy and 0.71 recall while maintaining 0.97 precision. The multiclass analysis further showed that while binary detection works well, rare attack types (R2L, U2R) require specialized approaches — a finding that would not have emerged from binary metrics alone.

From a stakeholder perspective, the most useful takeaway is not simply that one model scored higher than another. The project identifies a reproducible workflow for loading security data, exploring class structure, encoding mixed features, training class-weighted models, evaluating across multiple metrics, and turning those outputs into actionable recommendations for security operations. That makes the work suitable both as a classroom presentation and as a foundation for a more advanced intrusion-detection system.

References

Tavallaee, M., Bagheri, E., Lu, W., & Ghorbani, A. A. (2009). *A detailed analysis of the KDD CUP 99 data set*. 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications. https://doi.org/10.1109/CISDA.2009.5356528

hassan06. (n.d.). *NSL-KDD* [Data set]. Kaggle. https://www.kaggle.com/datasets/hassan06/nslkdd

Anthropic. (2026). *Claude* (claude-sonnet-4-6) [Large language model]. This project used Claude to assist with code development, analysis, and report writing. https://www.anthropic.com

Appendix

- Reproducible pipeline entry point: `python main.py --data-dir data/nsl-kdd --output-dir outputs`
- EDA notebook: `EDA.ipynb`
- Generated artifacts:
  - `outputs/eda_summary.json` — dataset summary statistics
  - `outputs/label_distribution.png` — top 10 fine-grained attack labels
  - `outputs/attack_category_distribution.png` — 5-category bar and pie charts
  - `outputs/binary_class_distribution.png` — normal vs attack counts
  - `outputs/correlation_heatmap.png` — feature correlation matrix
  - `outputs/numeric_feature_boxplot.png` — key features by attack category
  - `outputs/categorical_features_by_attack.png` — protocol/service/flag analysis
  - `outputs/confusion_matrices.png` — side-by-side binary confusion matrices
  - `outputs/roc_curves.png` — ROC curve comparison with AUC scores
  - `outputs/feature_importance.png` — top 20 Random Forest feature importances
  - `outputs/model_comparison.png` — metric comparison bar chart
  - `outputs/multiclass_confusion_matrix.png` — 5-class confusion matrix heatmap
  - `outputs/multiclass_report.txt` — per-class precision/recall/F1
  - `outputs/model_metrics.json` — binary metrics in JSON
  - `outputs/MODEL_METRICS.md` — binary metrics in Markdown